HDOC DAY-75

1. Assign explicit values starting from 10 in an enumeration and print them. Input: Nil, Output: A=10, B=11, C=12

Algorithm:

1. Start.
2. Define an enumeration Values.
3. Assign A = 10.
4. Let C automatically assign B = 11 and C = 12.
5. Print the values of A, B, and C.
6. Stop.

- # C Program:

```
#include <stdio.h>

enum Values
{
   A = 10,
   B,
   C
};


int main()
{
   printf("A = %d\n", A);
   printf("B = %d\n", B);
   printf("C = %d\n", C);

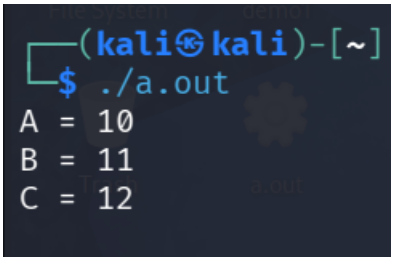

   return 0;
}
```

Test Case Table:

Test Case	Input	Expected Output	Actual Output	Result
1	Nil	A = 10 , B = 11 , C = 12	A = 10 , B = 11 , C = 12	Pass

Compilation and Execution:

The following test case has been compiled and executed successfully.



2. Use enumeration to represent menu choices (ADD, SUBTRACT, MULTIPLY) and perform operations using switch. Input: ADD 10 20, Output: 30

Algorithm

1. Start.
2. Define enumeration Menu with ADD = 1, SUBTRACT = 2, and MULTIPLY = 3.
3. Display the menu.
4. Read the user's choice.
5. Read two numbers.
6. Use switch to check the selected operation.
7. For ADD, add the two numbers.
8. For SUBTRACT, subtract the second number from the first.
9. For MULTIPLY, multiply the two numbers.
10. If the choice is invalid, display "Invalid choice".
11. Stop.

- ## C Program:

- #include <stdio.h>

```
enum Menu
{
   ADD = 1,
   SUBTRACT,
   MULTIPLY
};

int main()
{
   int choice;
   int num1, num2;

   printf("1. ADD\n");
   printf("2. SUBTRACT\n");
   printf("3. MULTIPLY\n");

   printf("Enter your choice: ");
   scanf("%d", &choice);

   printf("Enter two numbers: ");
   scanf("%d %d", &num1, &num2);

   switch(choice)
   {
      case ADD:
         printf("Result = %d\n", num1 + num2);
         break;

      case SUBTRACT:
         printf("Result = %d\n", num1 - num2);
         break;

      case MULTIPLY:
         printf("Result = %d\n", num1 * num2);
         break;

      default:
         printf("Invalid choice\n");
```

```
    }

    return 0;
}
```

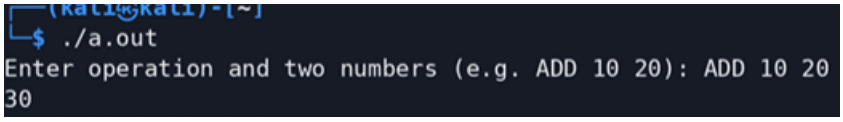
Test Case Table:

Test Case	Choice	Num1	Num2	Expected Output
1	(ADD)	10	20	Result = 30
2	(SUBTRACT)	50	15	Result = 35
3	(MULTIPLY)	6	7	Result = 42
4	(SUBTRACT)	5	12	-7
5	(MULTIPLY)	3	-4	-12
6	(DIVIDE)	8	2	Invalid Operation
7	(ADD)	5	*	Invalid numbers

Compilation and Execution:

The following program has been compiled and executed successfully.

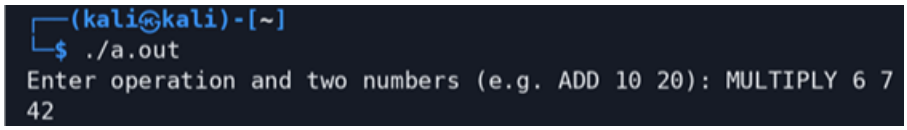
Test Case 1:



Test Case 2:



Test Case 3:



Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter operation and two numbers (e.g. ADD 10 20): SUBTRACT 5 12
-7
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter operation and two numbers (e.g. ADD 10 20): multiply 3 -4
-12
```

Test Case 6:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter operation and two numbers (e.g. ADD 10 20): DIVIDE 8 2
Invalid operation. Enter ADD, SUBTRACT or MULTIPLY.
```

Test Case 7:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter operation and two numbers (e.g. ADD 10 20): ADD 5 x
Invalid numbers. Enter two integers after the operation.
```